Lab 2
Controlling the 7-Segment Displays with Digital Logic

Ronald Nguyen
Nguyen.rona@northeastern.edu
Ayaan Gupta
Gupta.ayaa@northeastern.edu

Professor Thomas R. Consi
EECE 2160, Section 2

Submit Date: October 10, 2026
Due Date: October 10, 2026

Abstract

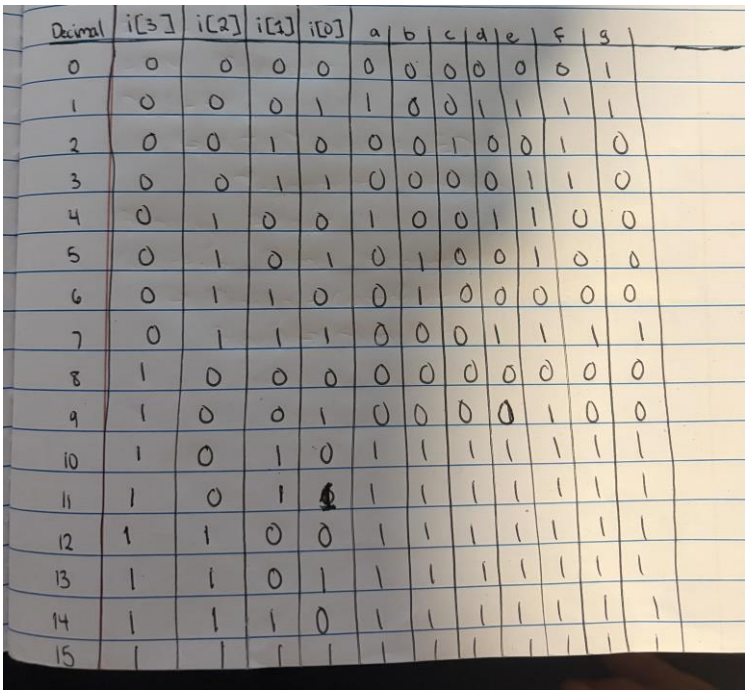
The purpose of this lab was to use digital logic to control the 7-segment displays on the DE1-SoC board. A Binary Coded Decimal (BCD) decoder was designed from Karnaugh map simplified Boolean expressions, simulated, and programmed onto the board. An enable input was then added to turn the display off on command, and finally the decoder was extended to display an 8-bit unsigned integer (0 to 255) across three displays with leading zeroes suppressed.

Introduction

A seven-segment display is made up of seven LED segments, labeled a through g, that can be lit in different combinations to show a single digit. On the DE1-SoC, each segment is active low, meaning a segment turns on when it receives a 0 and turns off when it receives a 1. This lab designed three circuits: a BCD decoder, a BCD decoder with an enable input, and an 8-bit to decimal decoder.

A BCD decoder takes a 4-bit input representing a decimal digit from 0 to 9 and outputs the seven segment values needed to display that digit. Since the circuit has four switch inputs, there are 16 possible input values, six of which (10-15) are not valid BCD numbers. These non-BCD inputs were not treated as don't cares. Instead, a 1 was entered for every segment whenever a non-BCD input occurred so that the display turns off for those inputs. The truth table for all 16 inputs was completed and is shown below in Table 1.

Table 1: Truth table for the 7-segment BCD decoder (0 = segment ON, 1 = segment OFF)



Decimal	i[3]	i[2]	i[1]	i[0]	a	b	c	d	e	f	g
0	0	0	0	0	0	0	0	0	0	0	1
1	0	0	0	1	1	0	0	1	1	1	1
2	0	0	1	0	0	0	1	0	0	1	0
3	0	0	1	1	0	0	0	0	1	1	0
4	0	1	0	0	1	0	0	1	1	0	0
5	0	1	0	1	0	1	0	0	1	0	0
6	0	1	1	0	0	1	0	0	0	0	0
7	0	1	1	1	0	0	0	1	1	1	1
8	1	0	0	0	0	0	0	0	0	0	0
9	1	0	0	1	0	0	0	0	1	0	0
10	1	0	1	0	1	1	1	1	1	1	1
11	1	0	1	1	1	1	1	1	1	1	1
12	1	1	0	0	1	1	1	1	1	1	1
13	1	1	0	1	1	1	1	1	1	1	1
14	1	1	1	0	1	1	1	1	1	1	1
15	1	1	1	1	1	1	1	1	1	1	1

From the truth table, seven Karnaugh maps were used to find a minimized Boolean expression for each segment output. The complete Karnaugh map analysis for the e segment is shown in Figure 1. For segment e, every input where i[0] = 1 produces a 1, which forms a group of eight cells and simplifies to i[0]. The remaining 1s at inputs 4 and 12 were grouped with inputs 5 and 13 to form i[2]i[1]′, and the non-BCD inputs 10, 11, 14, and 15 were grouped to form i[3]i[1].

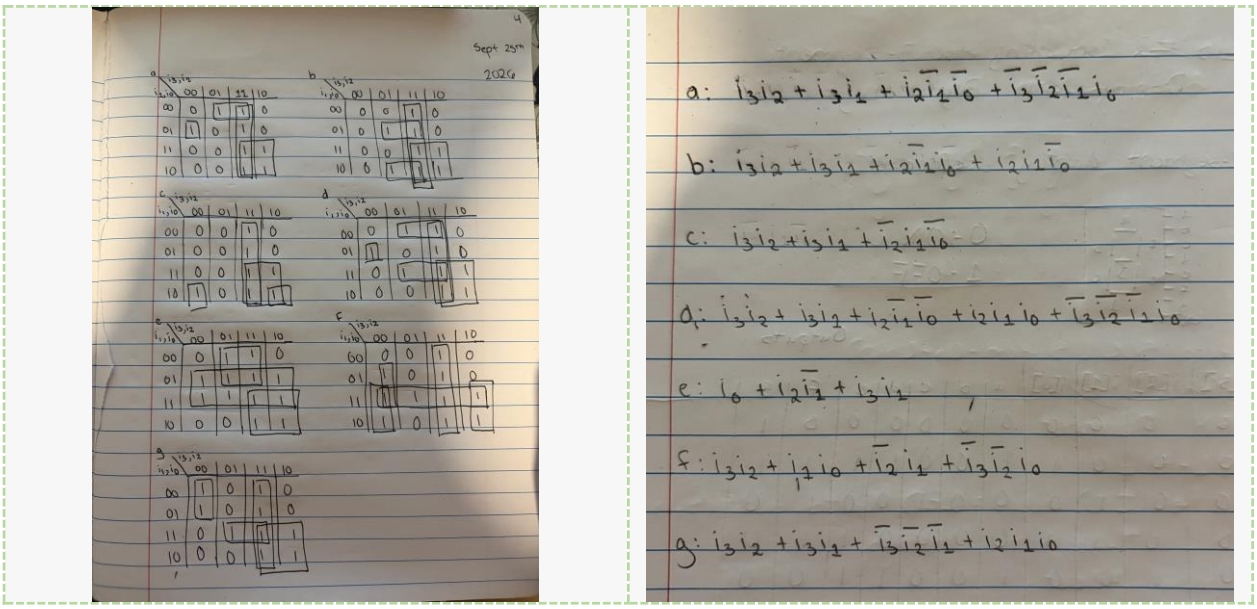


Figure 1: Karnaugh maps and derived Boolean Expressions

Setup

This lab was completed on a lab computer running Quartus Prime Lite, connected to an Intel DE1-SoC board. Each circuit was designed in the Quartus schematic editor, tested using a University Program VWF functional simulation, and then programmed onto the FPGA through the Quartus Programmer. Slide switches SW[0] through SW[7] and SW[9] were used as inputs, and the HEX0, HEX1, and HEX2 7-segment displays were used as outputs. For the final part of the lab, the LPM_DIVIDE, LPM_CONSTANT, and LPM_COMPARE components were added to the project from the Quartus IP Catalog.

Design

The BCD Decoder

The display7 circuit has four inputs, i[3] through i[0], and seven outputs, O[6] through O[0]. Each output was built directly from its Boolean expression using AND, OR, XOR, and NOT gates, as shown in Figure 2. In this design, O[6] represents segment a and O[0] represents segment g, so O[6] was assigned to HEX0[0] (segment a) and O[0] was assigned to HEX0[6]

(segment g). The inputs i[3], i[2], i[1], and i[0] were assigned to SW[3], SW[2], SW[1], and SW[0].

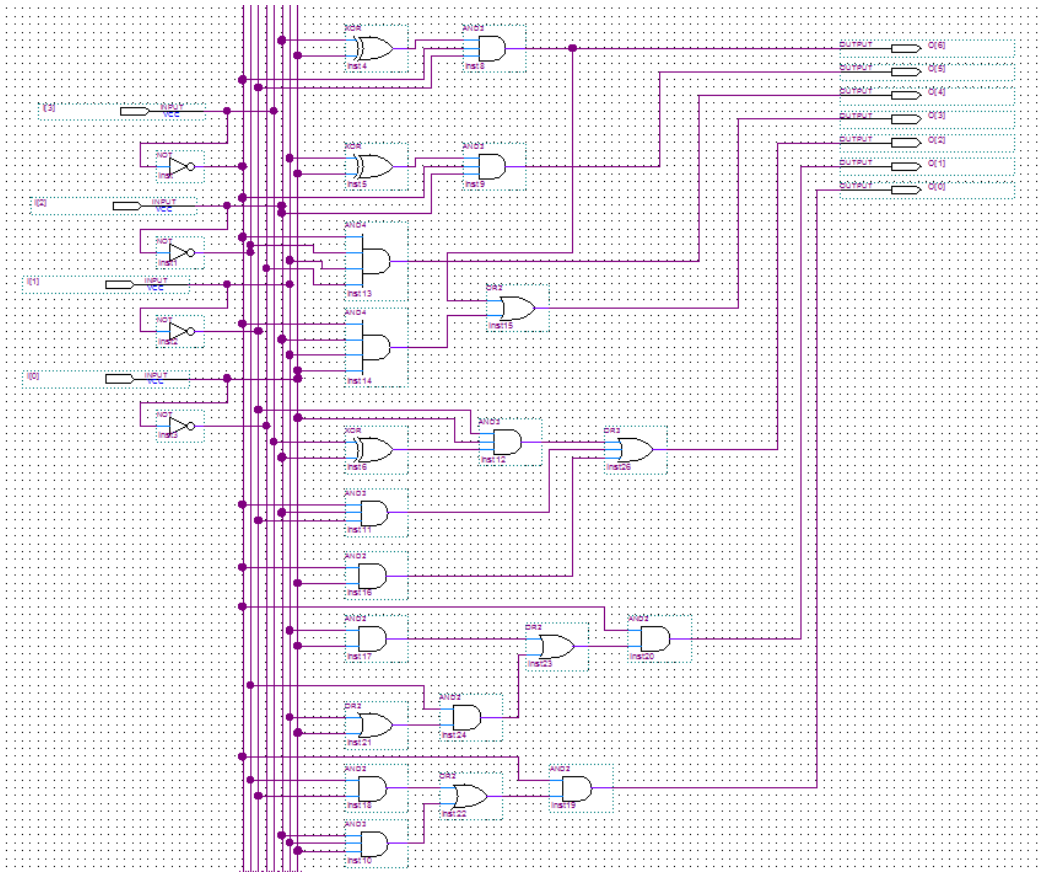


Figure 2: Schematic of the display7 BCD decoder circuit

The Enable Input

The display7_en circuit uses the display7 block as a single component and adds an enable input on SW[9], as shown in Figure 3. The enable input passes through a NOT gate, and the inverted signal is ORed with each of the seven display7 outputs. When enable is 1, the NOT gate outputs a 0, so each OR gate simply passes the display7 output through and the display works as usual. When enable is 0, the NOT gate outputs a 1, which forces every OR gate output to 1 and turns every segment off regardless of the other inputs.

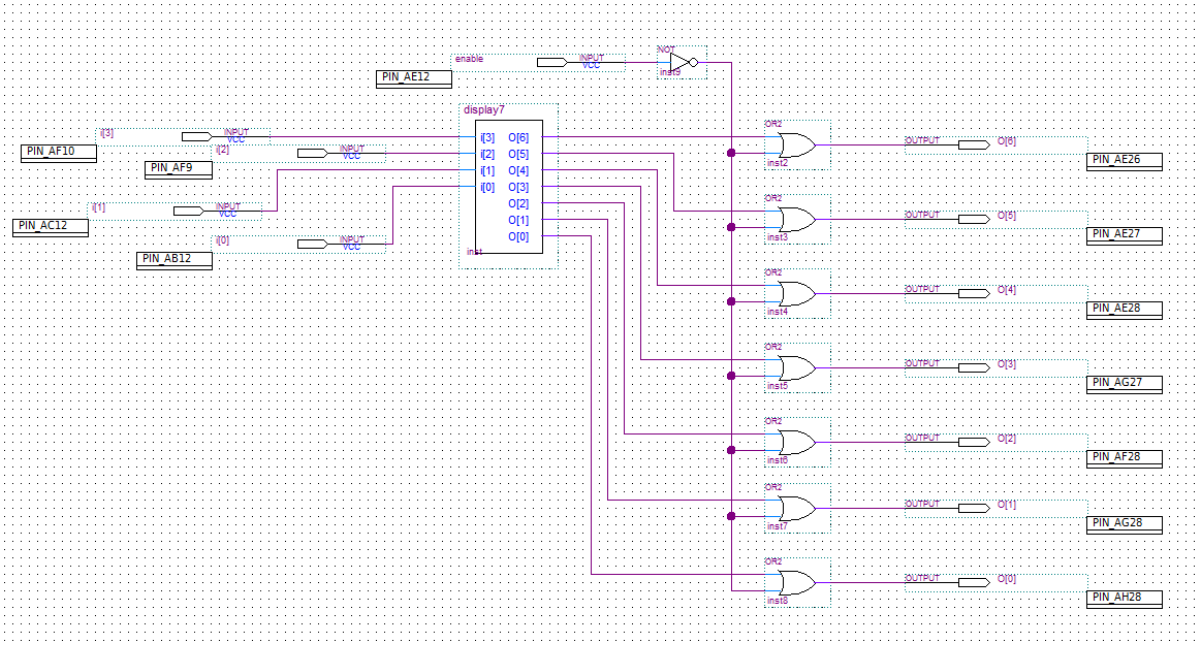


Figure 3: Schematic of the display7_en circuit with the enable input on SW[9]

The 8-Bit Decimal Display

The display_8bits circuit takes an 8-bit unsigned integer from SW[7..0] and displays its decimal value (0 to 255) on HEX2, HEX1, and HEX0, as shown in Figure 4. The circuit works by repeatedly dividing the input by 10.

The first divider divides SW[7..0] by 10. Its remainder, r1, is always between 0 and 9, so only its lower four bits, r1[3..0], are needed and are connected to the inputs of the HEX0 display7_en block. The quotient of the first division, q1, is then passed into the numerator of the second divider. The remainder of the second division, r2, is the tens digit and is connected to the HEX1 display7_en block, and the quotient of the second division, q2, is the hundreds digit and is connected to the HEX2 display7_en block. For example, an input of 127 gives 127 / 10 = 12 R 7, then 12 / 10 = 1 R 2, so HEX2 shows 1, HEX1 shows 2, and HEX0 shows 7.

To suppress leading zeroes, two LPM_COMPARE blocks (comp0) were used, each configured to check whether its 8-bit input is greater than a constant 0. HEX0 has its enable tied to VCC so that it always displays its digit, even when the result is 0. The first comparator checks q1, and its

agb output enables HEX1, so HEX1 only turns on when the input is 10 or greater. The second comparator checks q2 and enables HEX2, so HEX2 only turns on when the input is 100 or greater. HEX1 is enabled using q1 rather than its own digit r2 so that a middle zero is still shown for numbers like 205, where 205 / 10 = 20 R 5 and 20 / 10 = 2 R 0. Since q1 = 20 is greater than 0, HEX1 stays on and correctly displays 0.

The inputs and outputs were consolidated into buses so the circuit can be reused as a single block in future labs. The switches were grouped into a single SW[7..0] input bus, and the seven outputs of each display7_en block were connected by name to the HEX0[6..0], HEX1[6..0], and HEX2[6..0] output buses. Individual wires were tapped off of each bus by naming them after the bus and the bit they represent, such as r1[0] or HEX0[3].

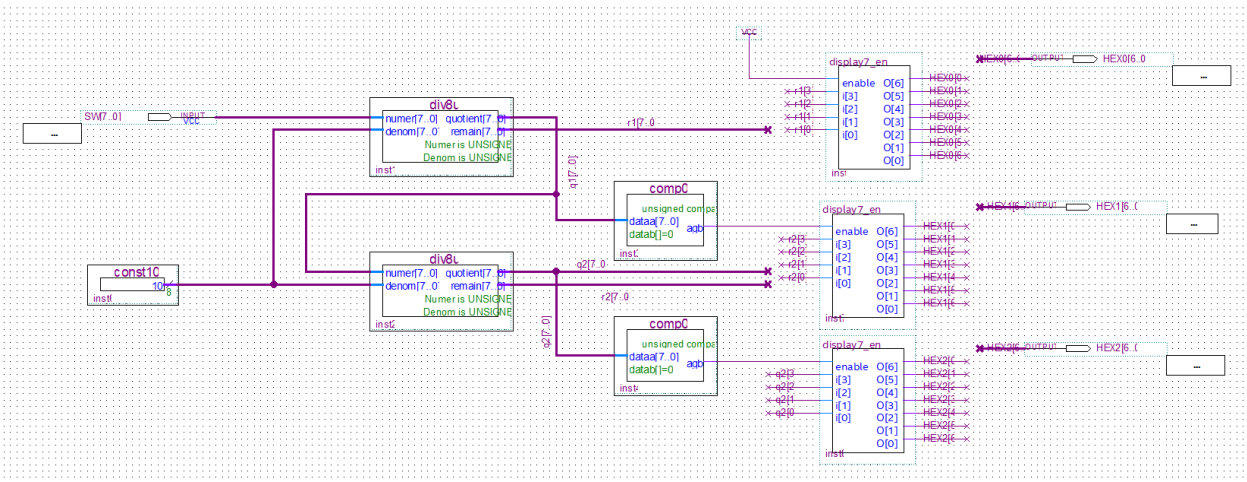


Figure 4: Schematic of the display_8bits circuit

Results & Analysis

Simulating the BCD Circuit

After the display7 schematic was compiled, a functional simulation was run to confirm its behavior before testing it on the board. Three BCD inputs, 0001 (1), 0101 (5), and 1000 (8), were applied, followed by the non-BCD input 1100 (12). As shown in Figure 5, each BCD input

produced the expected 7-segment output from Table 1. For the non-BCD input 1100, every output went to 1, turning the whole display off as intended.

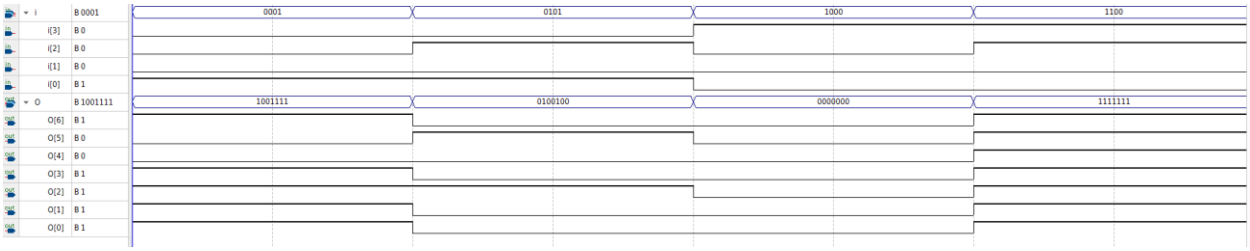


Figure 5: Simulation results for the display7 BCD decoder, with each input and output bit shown

Testing the BCD System on the DE1-SoC Board

Once the simulation matched the expected truth table, the design was compiled again with the pin assignments and programmed onto the DE1-SoC. SW[3] through SW[0] were used as the 4-bit input and HEX0 was used as the output. As shown in Figure 6, the display correctly showed 8, 7, and 5 for their matching switch inputs. and the display turned off when a non-BCD input was entered. However, when the non-BCD input was entered, the display stayed lit instead of turning off. This was traced back to the Karnaugh maps, where the non-BCD inputs had been treated as don’t cares instead of being set to 1. The Karnaugh maps and Boolean expressions were corrected, and the updated design was verified in simulation, where the non-BCD input 1100 produced 1111111 and turned every segment off, as shown in Figure 5. The image in Figure 6 was taken before this correction, and the corrected design could not be retested on the board before the submission deadline.

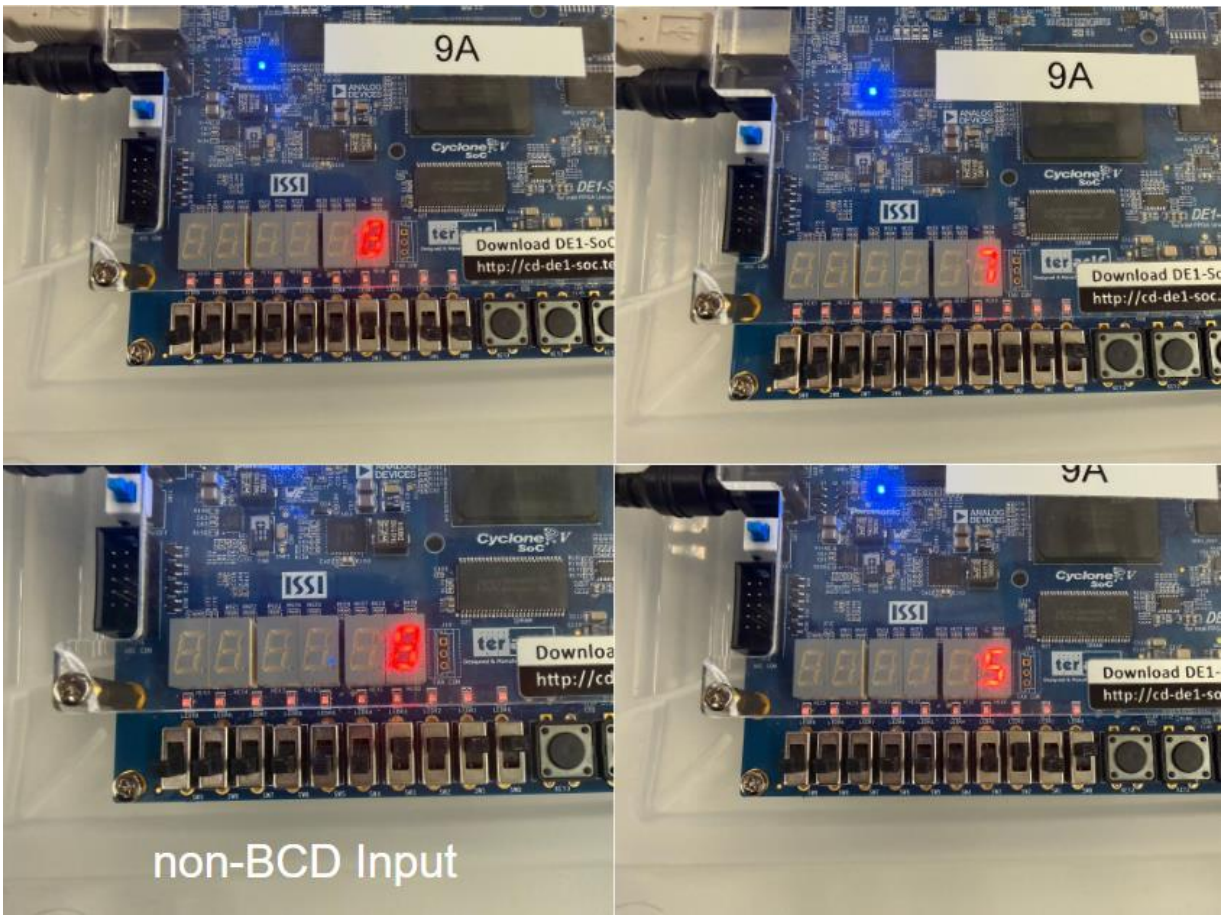


Figure 6: Board results for the display7 BCD decoder with three decimal inputs and one non-BCD input

Enabling the 7-Segment Display

The display7_en circuit was then programmed onto the board with SW[9] as the enable input. With the enable switch on, the display behaved the same as the display7 circuit and showed the digit selected by SW[3] through SW[0], as shown in Figure 7. With the enable switch off, the display stayed blank no matter what the other switches were set to, as shown in Figure 8, confirming that the OR gates were forcing every segment off.

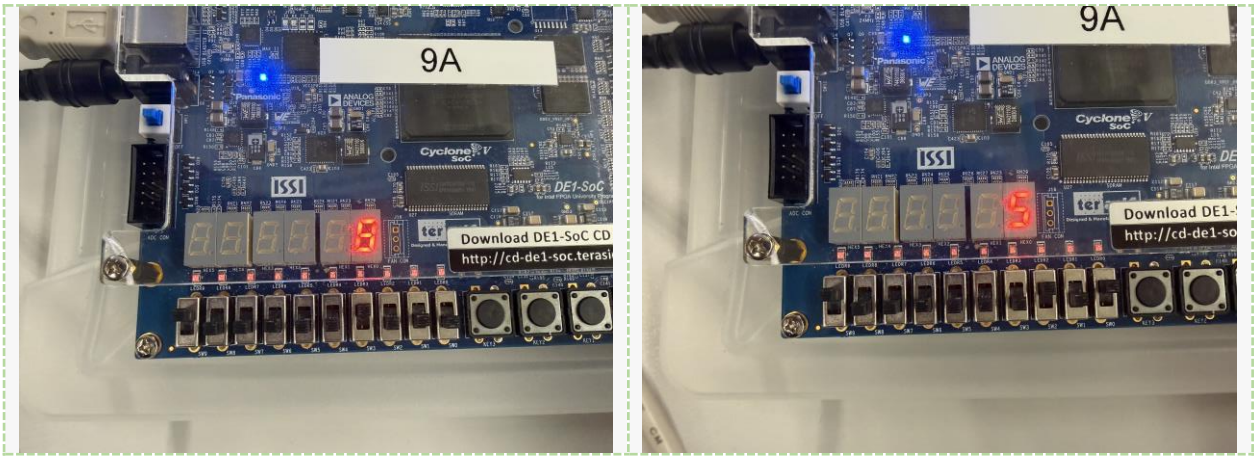


Figure 7: display7_en output with the enable switch on

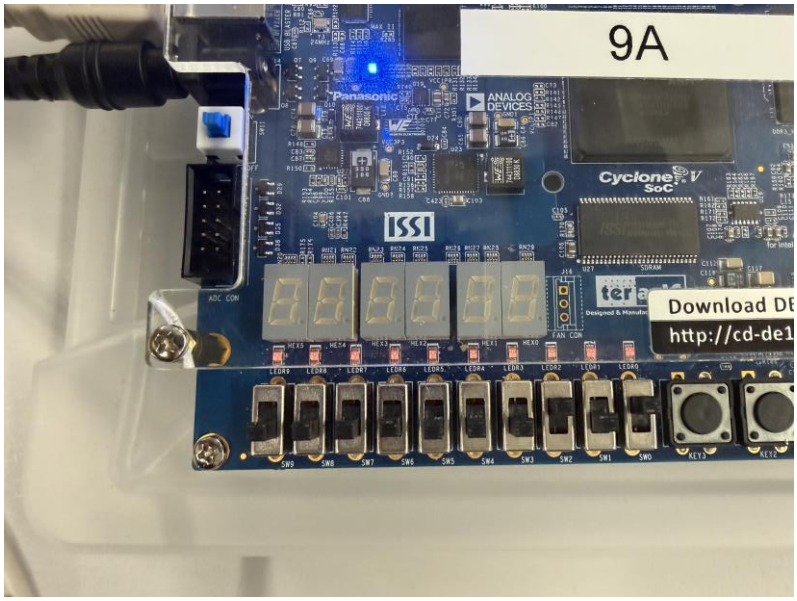


Figure 8: display7_en output with the enable switch off

Displaying an 8-Bit Unsigned Integer

Finally, the display_8bits circuit was programmed onto the board and tested with three inputs that require one, two, and three decimal digits. With an input of 00000111 (7), only HEX0 turned on and displayed 7, while HEX1 and HEX2 stayed off, as shown in Figure 9. With an input of 00010011 (19), HEX0 displayed 9, HEX1 was enabled and displayed 1, while HEX2 stayed off as shown in Figure 10. The first two cases confirmed that leading zeroes are suppressed, and the third confirmed that all three digits are displayed when the input is 100 or greater.

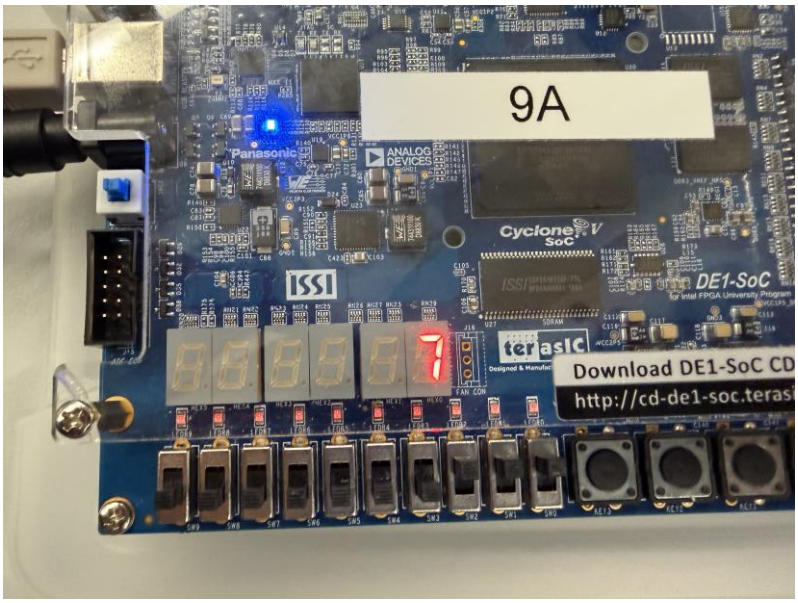


Figure 9: display_8bits output for a 1-digit input

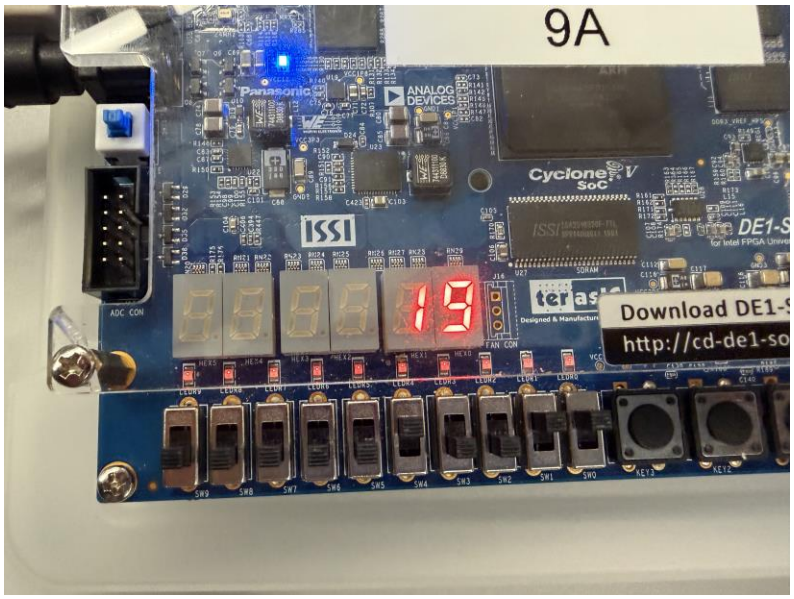


Figure 10: display_8bits output for a 2-digit input

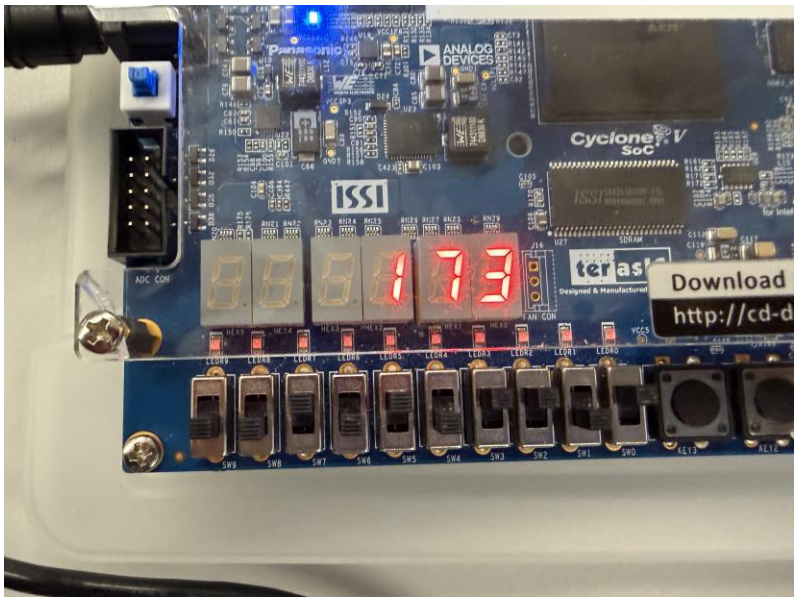


Figure 11: display_8bits output for a 3-digit input

Conclusion

The purpose of this lab was to control the 7-segment displays on the DE1-SoC board using digital logic designed in Quartus Prime. A BCD decoder was built from Karnaugh map simplified Boolean expressions, with non-BCD inputs set to turn the display off, an enable input was added to blank the display on command, and the decoder was extended to display 8-bit unsigned integers across three displays with leading zeroes suppressed. Each circuit was simulated and then verified in hardware. Packing each circuit into a symbol file and reusing it in the next part of the lab, along with using pre-made LPM components for division and comparison, reinforced how larger digital systems can be built out of smaller, reusable blocks.

References

[1] “Lab 2, Controlling the 7-Segment Displays with Digital Logic,” Northeastern University, EECE 2160, Fall 2026.